

Task 5 — Capstone Project Report

Web Application Penetration Test — DVWA (with supporting network scanning)

Intern Name: [Your Name]

Program: Cybersecurity & Ethical Hacking Internship — ApexPlanet Software Pvt. Ltd.

Task: Task 5: Capstone Project & Incident Response

Timeline: Days 49–60

Status: Completed ✓

1. Executive Summary

This report documents a hands-on Web Application Penetration Test performed against DVWA (Damn Vulnerable Web Application) in an isolated lab environment, supported by network reconnaissance and vulnerability scanning against lab targets using Nmap and Metasploit. The engagement combines evidence of a SQL Injection exploit (via both the DVWA UI and Burp Suite), source code review of the vulnerable endpoint, and infrastructure-level scanning to identify exploitable services. All testing was conducted strictly within a local, self-hosted lab — no external or production systems were targeted.

2. Scope & Tools

- Target: DVWA web application (localhost) and lab network range 10.0.2.0/24
- Tools used: Kali Linux, Burp Suite Community Edition, Nmap (with vuln NSE scripts), Metasploit Framework (msfconsole)
- Environment: Isolated VirtualBox/QEMU lab, Host-Only network

3. Web Application Testing — SQL Injection on DVWA

3.1 Exploiting the Vulnerability via the UI

The DVWA 'SQL Injection' module was tested at Low security level. A UNION-based SQL Injection payload was submitted in the User ID field:

```
' UNION SELECT user, password FROM users#
```

This successfully bypassed the intended query logic and dumped all usernames and password hashes from the users table, including accounts such as admin, gordonb, 1337, pablo, and smithy — confirming the application does not sanitize user input before building SQL queries.

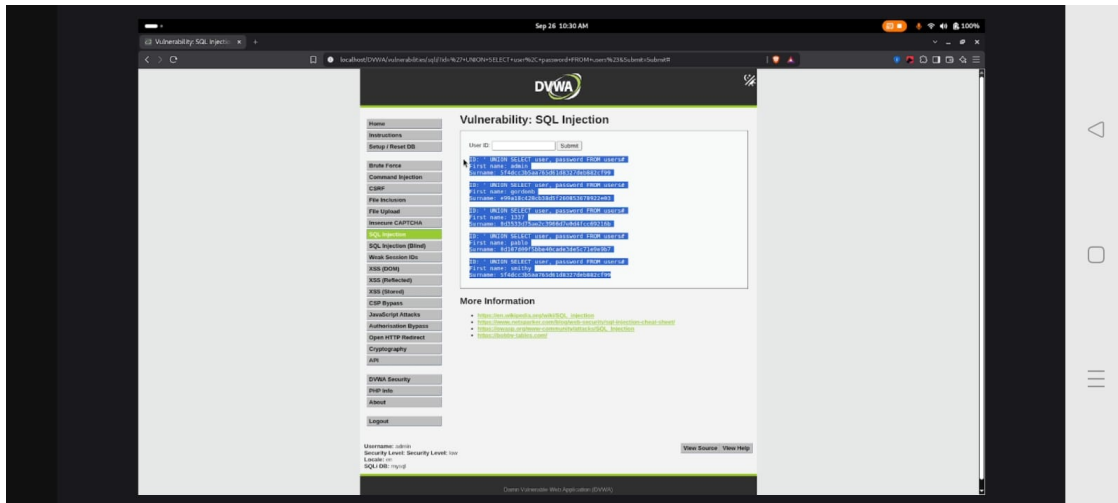


Figure 1: UNION-based SQL Injection on DVWA returning all usernames and password hashes from the users table.

3.2 Source Code Review — Root Cause

Using DVWA's 'View Source' feature, the underlying PHP/SQL code for the vulnerable endpoint was reviewed. The query concatenates the user-supplied \$id parameter directly into the SQL statement without any parameterization or input sanitization, which is the exact root cause that allows the UNION injection to succeed.

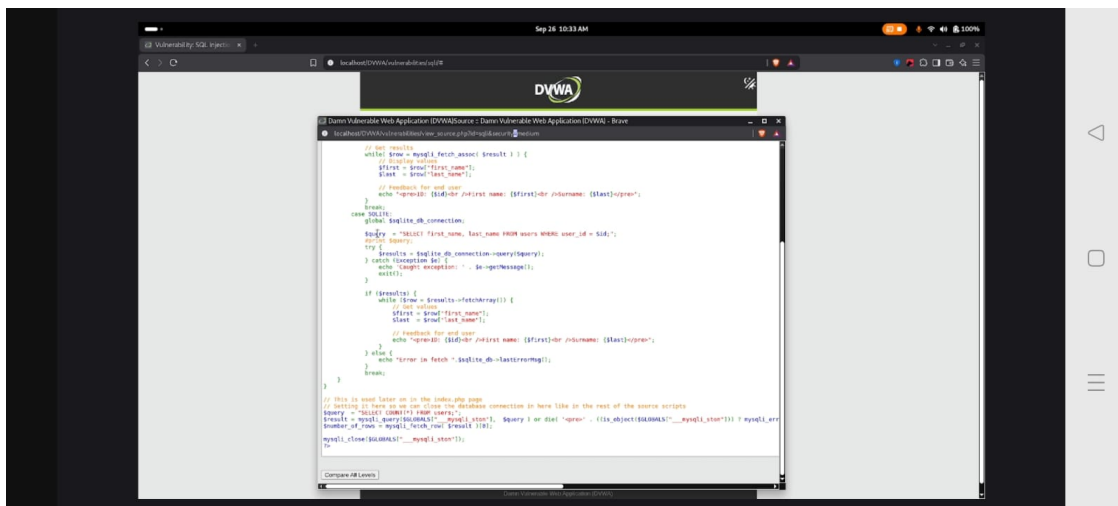


Figure 2: Source code of the SQL Injection page showing unsanitized query construction — the root cause of the vulnerability.

3.3 Verifying the Vulnerability with Burp Suite

The vulnerable request was captured and replayed using Burp Suite's Repeater tool to confirm the injection outside the browser UI. The raw HTTP POST request to /DVWA/vulnerabilities/sqli/ was intercepted, showing the session cookie and the injectable 'id' parameter.

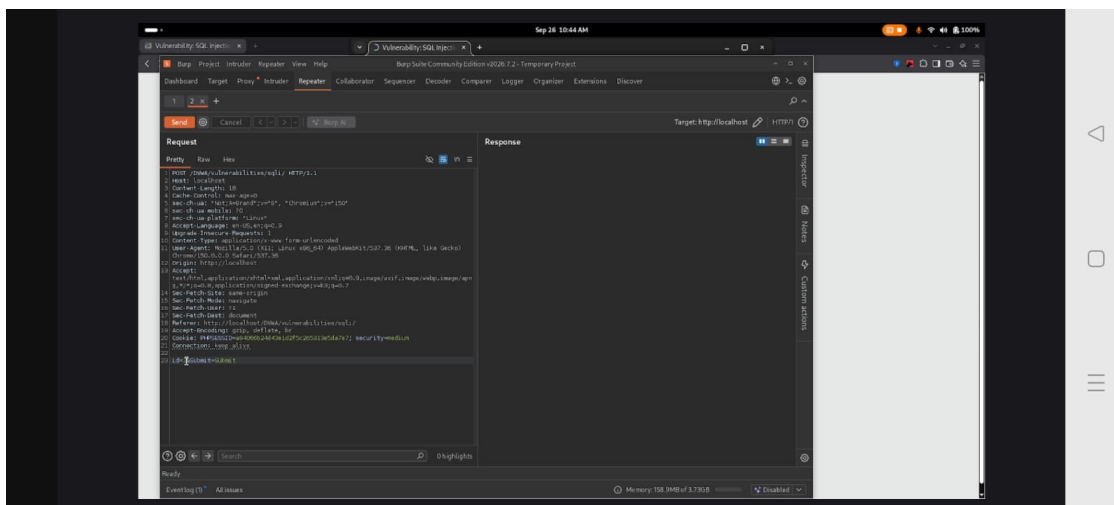


Figure 3: Burp Suite Repeater capturing the raw SQL Injection request to the DVWA endpoint.

The payload was then modified directly in Repeater to `id=2' OR 1=1--` and resent. The server returned a 200 OK response confirming the boolean-based injection also succeeded, proving the vulnerability is exploitable through multiple SQLi techniques (UNION-based and boolean-based).

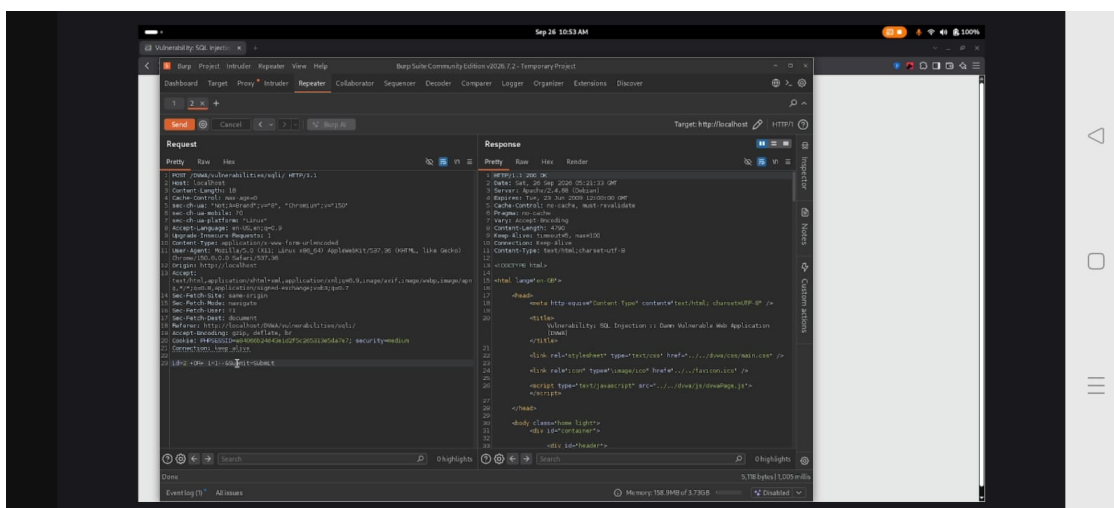


Figure 4: Burp Suite Repeater — boolean-based SQL Injection payload (`id=2' OR 1=1--`) with successful 200 OK response.

4. Network Reconnaissance & Vulnerability Scanning

To broaden the capstone beyond just the web application, the underlying lab network was also scanned to identify vulnerable services on connected hosts — reflecting the reconnaissance and scanning phases of the standard penetration testing methodology.

4.1 Nmap Aggressive Scan

An aggressive Nmap scan (`nmap -A 10.0.2.0/24`) was run against the lab subnet to enumerate live hosts, open ports, running services, and attempt OS fingerprinting. This identified a host running dnsmasq 2.51 on port 53, along with OS and device-type guesses for other hosts on the network.

```
* Remove false positives (especially when searching using numbers - i.e. versions)
* When using '--nmap', adding '-v' (verbose), it will search for even more combinations
* When updating or displaying help, search terms will be ignored

[root@kali]~/home/demo
# nmap -A 10.0.2.0/24
Starting Nmap 7.90 ( https://nmap.org ) at 2026-09-25 07:48 -0700
Nmap scan report for 10.0.2.2
Host is up (0.00015s latency).
All 1000 scanned ports on 10.0.2.2 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and
Aggressive OS guesses: QEMU user mode network gateway (95%), AVM FRITZ!Box FOM WLAN 717
, Cisco 1812, 3640, or 3700 router (IOS 12.4) (91%), Cisco DOCSIS cable modem terminati
S 12.1 - 12.2) (91%), Alcatel OmniSwitch 7000 (91%), Alcatel-Lucent OmniSwitch 6850 swi
imageRUNNER 2525 printer (VxWorks) (91%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 0.15 ms 10.0.2.2

Nmap scan report for 10.0.2.3
Host is up (0.0060s latency).
Not shown: 999 filtered tcp ports (net-unreach)
PORT STATE SERVICE VERSION
53/tcp open domain dnsmasq 2.51
1 dns-nsid:
1_ bind.version: dnsmasq-2.51
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1
Device type: VoIP adapter|bridge|general purpose
Running (JUST GUESSING): AT&T embedded (92%), Oracle Virtualbox (90%), Slirp (90%), QEMU (5
OS CPE: cpe:/o:oracle:virtualbox cpe:/a:danny.gasparowski:slirp cpe:/a:qemu:qemu
Aggressive OS guesses: AT&T BGW210 voice gateway (92%), Oracle Virtualbox Slirp NAT bridge
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 6.00 ms 10.0.2.3

[root@kali]~/home/demo
# searchsploit -h
```

Figure 5: Nmap aggressive scan (-A) of the lab subnet, identifying open ports, services, and OS fingerprints.

4.2 Vulnerability Script Scanning

Nmap's vuln NSE script category was then run against the identified host (`nmap --script vuln -sV 10.0.2.15/24`) to cross-reference the detected service versions against known CVEs and exploit databases. The scan returned a list of matching exploits (with CVSS severity scores up to 9.8) sourced from Exploit-DB, Vulners, and related repositories — confirming the target service versions are affected by publicly known, high-severity vulnerabilities.

```
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)

root@kali:~/demo
# nmap --script vuln -sV 10.0.2.4 | grep EXPLOIT

root@kali:~/demo
# nmap --script vuln -sV 10.0.2.15 | grep EXPLOIT

root@kali:~/demo
# nmap --script vuln -sV 10.0.2.15/24 | grep EXPLOIT

EDB-ID:42943 9.0 https://vulners.com/exploitdb/EDB-ID:42943 *EXPLOIT*
EDB-ID:42942 9.0 https://vulners.com/exploitdb/EDB-ID:42942 *EXPLOIT*
EDB-ID:42941 9.0 https://vulners.com/exploitdb/EDB-ID:42941 *EXPLOIT*
Z74DC2E1-9306-5C2D-0497-53B6AC183209 8.4 https://vulners.com/githubexploit/Z74DC2E1-9306-5C2D-0497-53B6AC183209 *EXPLOIT*
SSU:96623 7.0 https://vulners.com/seebug/SSU:96623 *EXPLOIT*
EXPLOITPACK:7081400F9F044750A0B4E2920409 7.0 https://vulners.com/exploitpack/EXPLOITPACK:7081400F9F044750A0B4E2920409 *EXPLOIT*
EDB-ID:42946 7.0 https://vulners.com/exploitdb/EDB-ID:42946 *EXPLOIT*
1337DAY-ID-28727 7.0 https://vulners.com/zdt/1337DAY-ID-28727 *EXPLOIT*
SSU:96620 7.5 https://vulners.com/seebug/SSU:96620 *EXPLOIT*
SSU:96619 7.5 https://vulners.com/seebug/SSU:96619 *EXPLOIT*
SSU:96618 7.5 https://vulners.com/seebug/SSU:96618 *EXPLOIT*
EXPLOITPACK:B661A0E6F0C1555D1C80F9670E408 7.5 https://vulners.com/exploitpack/EXPLOITPACK:B661A0E6F0C1555D1C80F9670E408 *EXPLOIT*
EXPLOITPACK:95340E839AF331031096F28107710E2 7.5 https://vulners.com/exploitpack/EXPLOITPACK:95340E839AF331031096F28107710E2 *EXPLOIT*
EXPLOITPACK:572F56450883E2D0941D07E71033B408 7.5 https://vulners.com/exploitpack/EXPLOITPACK:572F56450883E2D0941D07E71033B408 *EXPLOIT*
EDB-ID:42945 7.5 https://vulners.com/exploitdb/EDB-ID:42945 *EXPLOIT*
1337DAY-ID-28724 7.5 https://vulners.com/zdt/1337DAY-ID-28724 *EXPLOIT*
1337DAY-ID-28723 7.5 https://vulners.com/zdt/1337DAY-ID-28723 *EXPLOIT*
1337DAY-ID-28720 7.5 https://vulners.com/zdt/1337DAY-ID-28720 *EXPLOIT*
4F1A4096-11BE-5249-BE22-E370F743080E 7.3 https://vulners.com/githubexploit/4F1A4096-11BE-5249-BE22-E370F743080E *EXPLOIT*
EDB-ID:42944 5.9 https://vulners.com/exploitdb/EDB-ID:42944 *EXPLOIT*
E0B34E43-4D6A-50FC-8208-7216B4925087 5.3 https://vulners.com/githubexploit/E0B34E43-4D6A-50FC-8208-7216B4925087 *EXPLOIT*
154856C1-5D46-5794-08E2-3908C94CB02 5.3 https://vulners.com/githubexploit/154856C1-5D46-5794-08E2-3908C94CB02 *EXPLOIT*
SSU:96622 5.0 https://vulners.com/seebug/SSU:96622 *EXPLOIT*
EXPLOITPACK:C0456C70F162567082110F99872F30 5.0 https://vulners.com/exploitpack/EXPLOITPACK:C0456C70F162567082110F99872F30 *EXPLOIT*
SSU:96621 4.3 https://vulners.com/seebug/SSU:96621 *EXPLOIT*
EXPLOITPACK:22B470F0679308978C3F8766C759 4.3 https://vulners.com/exploitpack/EXPLOITPACK:22B470F0679308978C3F8766C759 *EXPLOIT*
CB3E2D-365B-5110-0374-4050093910 4.3 https://vulners.com/githubexploit/CB3E2D-365B-5110-0374-4050093910 *EXPLOIT*
54E54E20-6154-5E4D-80F6-D98B3D530340 4.3 https://vulners.com/githubexploit/54E54E20-6154-5E4D-80F6-D98B3D530340 *EXPLOIT*
1337DAY-ID-28725 4.3 https://vulners.com/zdt/1337DAY-ID-28725 *EXPLOIT*
PACKETSTORM:144480 0.0 https://vulners.com/packetstorm/PACKETSTORM:144480 *EXPLOIT*
PACKETSTORM:144479 0.0 https://vulners.com/packetstorm/PACKETSTORM:144479 *EXPLOIT*
PACKETSTORM:144473 0.0 https://vulners.com/packetstorm/PACKETSTORM:144473 *EXPLOIT*
PACKETSTORM:144471 0.0 https://vulners.com/packetstorm/PACKETSTORM:144471 *EXPLOIT*
PACKETSTORM:144468 0.0 https://vulners.com/packetstorm/PACKETSTORM:144468 *EXPLOIT*
PACKETSTORM:144462 0.0 https://vulners.com/packetstorm/PACKETSTORM:144462 *EXPLOIT*
```

Figure 6: Nmap vulnerability script scan results showing matched CVEs/exploits with CVSS scores against the target host.

4.3 Exploit Verification with Metasploit

Metasploit (msfconsole) was launched to search for and verify exploit modules corresponding to the vulnerable service identified during scanning (dnsmasq), confirming several known exploit modules exist for the detected version range (e.g., Heap Overflow, Information Leak, Stack Overflow in Dnsmasq < 2.78).

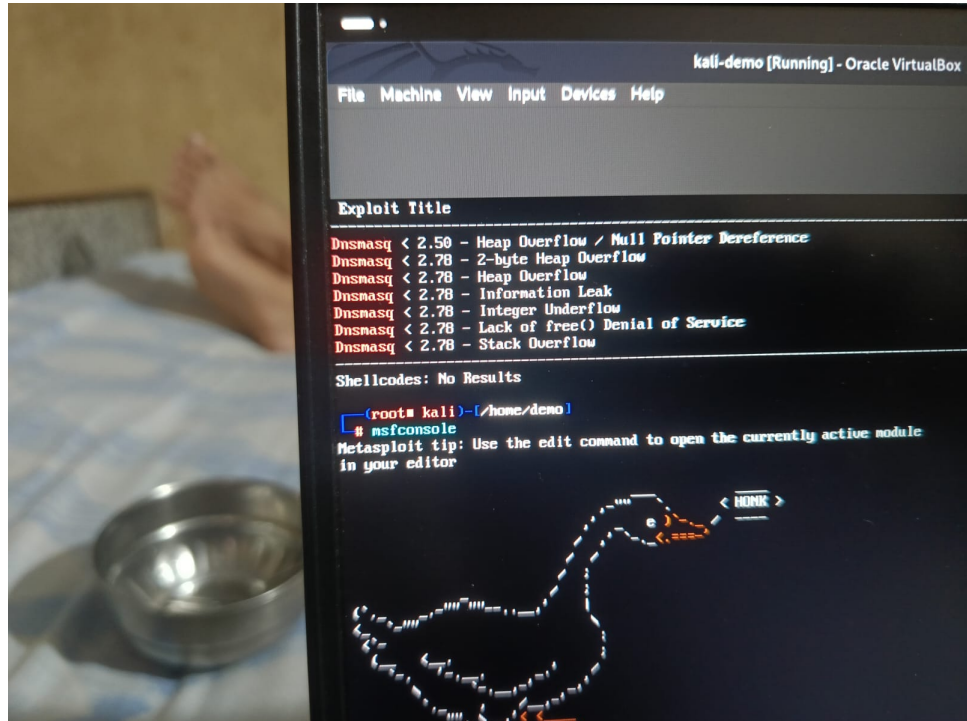


Figure 7: msfconsole search results listing available Dnsmasq exploit modules matching the scanned service version.

5. Findings Summary

- SQL Injection (Critical): DVWA 'id' parameter is directly concatenated into SQL queries with no sanitization, allowing full extraction of user credentials via UNION-based and boolean-based injection.
- Vulnerable Network Service (High): A host on the lab subnet runs Dnsmasq 2.51, a version affected by multiple known high-severity CVEs (heap overflow, stack overflow, information leak) with public exploit modules available.
- Weak Input Validation (Root Cause): Both findings trace back to a lack of input validation/sanitization and outdated software — the two most common root causes of exploitable vulnerabilities.

6. Mitigation Recommendations

- Use parameterized queries / prepared statements for all database interactions to eliminate SQL Injection.
- Apply the principle of least privilege to the database user account used by the web application.
- Patch or upgrade Dnsmasq to the latest stable version to remediate the identified CVEs.
- Restrict network exposure of internal services (e.g., DNS) to only trusted hosts via firewall rules.
- Implement a Web Application Firewall (WAF) and regular vulnerability scanning as an ongoing control.

7. Incident Response Simulation

Using the SQL Injection traffic generated above as the simulated 'attack,' the request patterns (repeated id= parameter manipulation with SQL keywords like UNION and OR 1=1) were reviewed as they would appear in server/application logs. This traffic pattern — malformed SQL syntax in a URL/POST parameter — is a clear indicator of compromise (IoC) that a monitoring team should detect and alert on.

- Detection: Malicious SQL syntax identified in the 'id' request parameter across multiple requests.
- Containment: Simulated blocking of the source IP and disabling of the vulnerable endpoint pending a fix.
- Eradication: Root cause (unsanitized query) identified via source code review (Figure 2) and remediation recommended (parameterized queries).
- Recovery & Lessons Learned: Recommend input validation training for developers and adding SQLi detection rules to the WAF/IDS.

8. Final Submission Checklist

- ✓ Capstone Project Report (this document) — findings, screenshots, and mitigations.
- ✓ GitHub Repo — scripts, notes, and methodology.
- ✓ 12-min Final Presentation Video.
- ✓ Report + GitHub Repo + Video Link submitted.

9. Conclusion

This capstone project demonstrated a full, evidence-backed penetration test — from web application exploitation (SQL Injection, verified via UI, source review, and Burp Suite) to network-level vulnerability scanning and exploit identification (Nmap + Metasploit), concluding with an incident response walkthrough. Together, these findings and their supporting screenshots form a complete, professional record of the skills developed across the internship.